

SUPSI

Postman clone (Krostman)

Students		
Antonio Marroffino		
Luca Fantò		
Degree Course	Module	Accademic Year
Bachelor Ingegneria Informatica	M-I6010 - C-I6012 - Applicazioni Web 2 - 2025	2024 - 2025
Stakeholders		Date
Patrick Ceppi		08/04/2025
Roberto Guidi		

Index

- **Introduction**
 - React framework
- **Problem**
 - Requirements analysis
- **State of the art**
- **Approach**
 - Technologies
 - Reusable components
 - Provider-based architecture
 - React-query
- **Results**
 - Static software demo
- **Conclusions**
 - Possible optimizations and improvements

Introduction

React framework



- **React:** JavaScript library developed by Facebook for dynamic user interfaces
- **Component-Based Architecture:** allows you to create modular and reusable components, facilitating management and scalability
- **Virtual DOM:** updates only the necessary parts of the page, improving performance
- **Ecosystem and Community:** offers extensive support with tools, libraries, and an active community for the development of modern applications

Problem

Requirements analysis



- **Integrable HTTP Client:** Create a component similar to PostMan, but customizable and integrable into various applications.
- **Integration Limitations:** Existing tools do not offer the necessary flexibility for specific customizations.
- **Specific Requirements:** Management of HTTP requests and request collections.
- **Configurability:** Need for a configurable and interactive component.

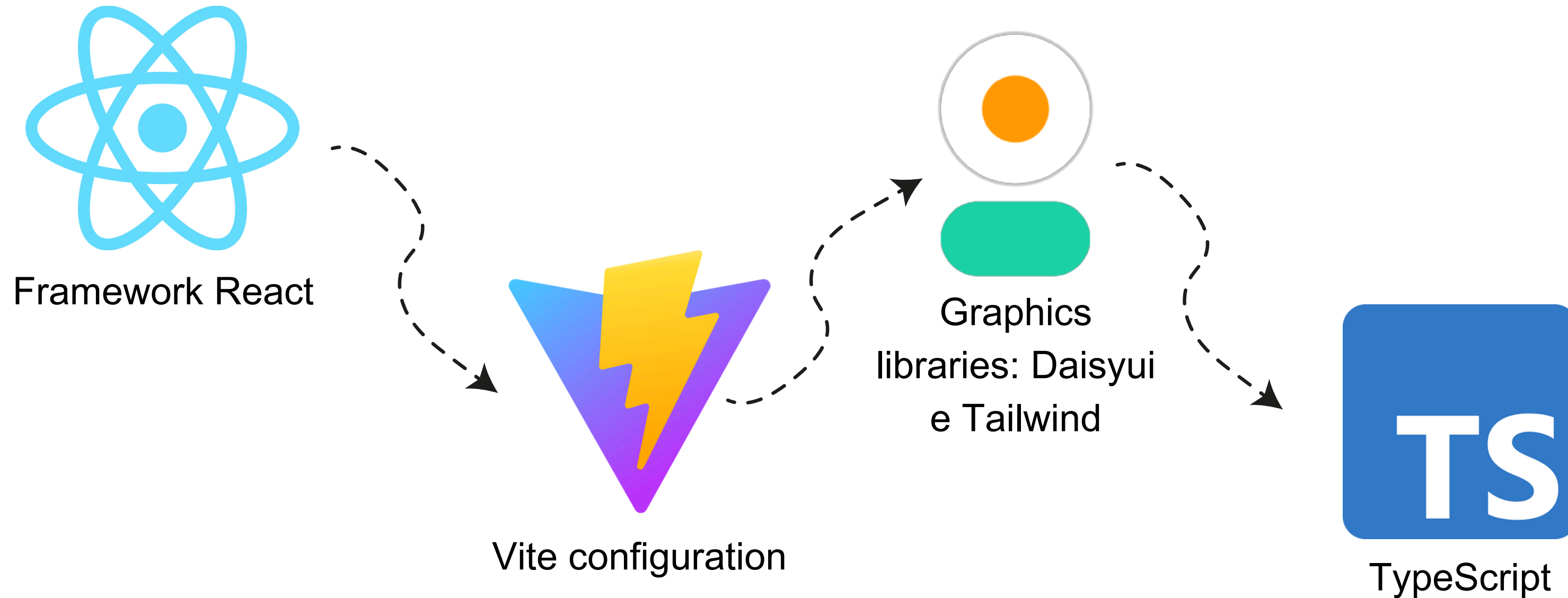
State of the art

- Send HTTP requests, test endpoints
- Create request collections, manage authentication, and support various data formats



Approach

Tecnologies



Approach

Reusable components

- **Configurability:** *url*
- **Customization:** *search* and *collections*
- **Interactivity:**
onResponseMessageClick



Reusability and Scalability

```
function App() {  
  return (  
    <QueryClientProvider client={queryClient}>  
      <HttpClient  
        url="https://supsi-ticket.cloudns.org/supsi-http-client"  
        search={true}  
        collections={true}  
        onResponseMessageClick={(response : HttpResponseDTO ) => alert(response.status)}  
      />  
    </QueryClientProvider>  
  );  
}
```

Approach

Provider-based architecture

- **Hierarchy:** allows child components to inherit state and functionality
- **Context Isolation:** separates logic from the components



Reusability, Scalability and Maintainability.

```
<UrlProvider url={url}>
  <CollectionsProvider>
    <SelectedRequestProvider>
      <RequestCRUDProvider>
        <FileProvider>
          <HttpProvider>
            <PreviewProvider>
              <PreviewRegistrar/>
              <div
                className="grid grid-cols-1 md:grid-cols-8 lg:grid-cols-16 h-screen overflow-hidden">
                {collections && (
                  <div className={`transition-all duration-300 ${sidebarClass} max-w-xs`}>
                    <Sidebar
                      showSearch={search}
                      isOpen={isSidebarOpen}
                      onToggle={toggleSidebar}
                    />
                  </div>
                )}
                <div className={` ${homeClass} max-w-full h-full overflow-auto`}>
                  <Home
                    collections={collections}
                    onResponseMessageClick={onResponseMessageClick}
                  />
                </div>
              </div>
            </PreviewProvider>
          </HttpProvider>
        </FileProvider>
      </RequestCRUDProvider>
    </SelectedRequestProvider>
  </CollectionsProvider>
</UrlProvider>
```


Approach

Contexts and Hooks

```
export type RequestCRUDContextType = {
  createRequest: (collectionId: number, requestCreationDTO: RequestCreationDTO) => Promise<RequestDTO>;
  updateRequest: (collectionId: number, requestId: string, requestDTO: RequestDTO) => Promise<RequestDTO>;
  deleteRequest: (collectionId: number, requestId: string) => Promise<void>;

  isCreatingRequest: boolean;
  isUpdatingRequest: boolean;
  isDeletingRequest: boolean;

  errorCreateRequest: Error | null;
  errorUpdateRequest: Error | null;
  errorDeleteRequest: Error | null;
}

export const RequestCRUDContext : Context<RequestCRUDContextType | undefined> = createContext<RequestCRUDContextType | undefined>(undefined);
```

Approach

React-query



- **Efficient Cache Management:** prevents repeated call
- **Request State Management:** Oversees and controls the various states of a request



Efficiency, Scalability and Reliability

```
export const useRequestsList : (collectionId: number) => UseQueryResult<...> = (collectionId: number) => {  
  const {url} = useUrl();  
  return useQuery<RequestDTO[], Error>({  
    queryKey: ['requests', collectionId],  
    queryFn: async () => {  
      const response :Response = await fetch(  
        `${url}/bff/collections/${collectionId}/requests?apiKey=${apiKey}`  
      );  
      if (!response.ok) throw new Error("Errore nel recupero delle richieste");  
      return response.json();  
    },  
    staleTime: 1000 * 60 * 5  
  });  
};
```

Result

Static software demo

Collections

Cerca...

Collection 1

POST

Land Beyond

DELETE

Withered Heath

POST

Land Beyond

GET

The Shire

PUT

Great River

POST

Grey Mountains

Import Collection

GET

https://supsi-ticket.cloudns.org/supsi-http-client/test-endpoints/fictional-character/image

Send

Save

Headers

Header	Value
Accept	image/png

Request Body

Raw content

200

41.30ms

6.15 KB

Call function

Preview

Raw



Conclusion

Possible optimizations and improvements

- **More reusable components**
- **Tests**
- **Improve responsivity**



THANKS FOR YOUR ATTENTION

